

Student Performance Analysis System

Objective / Aim

To design and implement a student performance analysis system in Java using **Object-Oriented Programming principles** to calculate marks, percentage, grade, highest and lowest marks, and overall academic performance.

Problem Statement

Develop a Java application for a college to analyze the academic performance of a student. The system should accept the student's details and marks. It should perform the following operations:

- Store subject marks using an array.
- Calculate total and average marks.
- Find the highest and lowest marks.
- Calculate percentage.
- Assign a grade based on percentage.
- Determine whether the student has passed or failed.
- Display the student's name in a formatted manner using String operations.
- Round the percentage to two decimal places using the Math package.
- Display a simple performance remark based on the student's percentage.
- The program should use a class to represent the student and demonstrate basic object-oriented programming.

Algorithm / Logic Description Create a class named Student. Declare the following data members:

- Roll number
- Student name
- Array of marks

Create a constructor to initialize the student details. Create methods to:

- Calculate total marks.
- Calculate average marks.
- Find highest marks.
- Find lowest marks.
- Calculate percentage.
- Determine grade.
- Display student details.

Store marks of five subjects in an integer array. Use a loop to calculate the total marks. Use Math.max() and Math.min() or suitable logic to determine the highest and lowest marks. Calculate the percentage from the total marks. Use Math.round() to display the percentage up to two decimal places. Use String operations such as trim(), toUpperCase(), and length() to process the student's name. Assign grades using the following criteria:

90–100	A+
80–89	A
70–79	B
60–69	C
50–59	D

Below 50 F (Fail) Consider the student passed only if the percentage is at least 50%. Display a performance remark based on the grade. Create a Student object in the main() method. Display the complete performance report.

Real-time application

College student performance analysis

Result

The **Student Performance Analysis System** was successfully implemented in Java using **OOP concepts, arrays, and built-in packages**. The program successfully calculates and displays the student's academic performance, including total marks, average, highest and lowest marks, percentage, grade, result, and performance remark.

✂ System Requirements

Hardware

Standard PC / Laptop
 Intel Core i3 / i5 processor or equivalent
 Minimum 4 GB RAM
 Minimum 500 MB free storage

Software

Operating System : Windows / Linux / macOS
 Java Development Kit : JDK 17 or JDK 21
 Text Editor or IDE : Eclipse, IntelliJ IDEA, NetBeans, or VS Code
 Command Prompt / Terminal

□ Algorithm / Logic Description

1. Create a class named Student.
2. Declare the following data members:
 - Roll number
 - Student name
 - Array of marks
3. Create a constructor to initialize the student details.
4. Create methods to:
 - Calculate total marks.
 - Calculate average marks.

- Find highest marks.
 - Find lowest marks.
 - Calculate percentage.
 - Determine grade.
 - Display student details.
5. Store marks of five subjects in an integer array.
 6. Use a loop to calculate the total marks.
 7. Use Math.max() and Math.min() (or suitable logic) to determine the highest and lowest marks.
 8. Calculate the percentage from the total marks.
 9. Use Math.round() to display the percentage up to two decimal places.
 10. Use String operations such as trim(), toUpperCase(), and length() to process the student's name.
 11. Assign grades using the following criteria:

90-100	-> A+
80-89	-> A
70-79	-> B
60-69	-> C
50-59	-> D
Below 50	-> F (Fail)
 12. Consider the student passed only if the percentage is at least 50%.
 13. Display a performance remark based on the grade.
 14. Create a Student object in the main() method.
 15. Display the complete performance report.

Real-time Application

College student performance analysis

1. Student.java — Base Class

Source Code

```
package basics;
import java.util.Scanner;

public class Student {
    public String name;
    protected int RollNo;
    protected int[] marks;

    public Student(String name, int RollNo, int[] marks) {
        this.name = name;
        this.RollNo = RollNo;
        this.marks = marks;
    }

    int getTotalMarks() {
        int total = 0;
```

```

        for (int i:marks) {
            total += i;
        }
        return total;
    }

    void percentage(){
        int total = getTotalMarks();
        double percent = (double) total / marks.length;
        System.out.println("Percentage: " + percent);
    }

    void displayDetails(){
        System.out.println("Name: " + name.toUpperCase());
        System.out.println("Roll Number: " + RollNo);
        System.out.println("Marks: ");
        for (int mark : marks) {
            System.out.print(mark + " ");
        }
        System.out.println();
        int total = getTotalMarks();
        System.out.println("Total Marks: " + total);
        percentage();
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter student name: ");
        String name = sc.next();

        System.out.print("Enter student roll number: ");
        int rollNo = sc.nextInt();

        System.out.print("Enter student marks: ");
        int[] marks = new int[3];
        for (int i = 0; i < 3; i++) {
            marks[i] = sc.nextInt();
        }

        Student s = new Student(name, rollNo, marks);
        s.displayDetails();

        sc.close();
    }
}

```

1. Concept and Exception in Simple Words

`Student` is the **base (parent) class**, declared `public` in package `basics` so it can later be extended from another package. Its fields use three different access levels on purpose:

Field	Access	Why
name	public	Visible from anywhere, including other packages
RollNo	protected	Visible in the same package AND to subclasses in other packages
marks	protected	Same as above — needed so a subclass elsewhere can use it

Its methods `getTotalMarks()`, `percentage()`, and `displayDetails()` are declared with **no access modifier** (package-private / default access) — meaning they are only usable from **within the `basics` package itself**. This matters a lot for the second program, explained below.

Exception in simple words: No exception is thrown or caught in this program. Two things worth knowing:

- `percentage()` divides `total` (an `int`) cast to `double` by `marks.length`. Because it's a **double division**, dividing by zero would *not* throw `ArithmeticException` — it would print `Infinity` or `NaN` instead. This program always creates a 3-element array, so it never hits zero anyway.
- `sc.nextInt()` would throw `InputMismatchException` (unchecked) if the user typed letters instead of a number for roll number or marks — the program does not guard against this.

2. Step-wise Execution

1. Scanner object is created.
2. User enters name (single word, via `sc.next()`).
3. User enters roll number (`int`).
4. A 3-element `int` array 'marks' is created.
5. A for loop reads 3 mark values from the user into the array.
6. A Student object 's' is created using the constructor, which stores name, RollNo, and marks.
7. `s.displayDetails()` is called:
 - a. `name.toUpperCase()` is printed.
 - b. RollNo is printed.
 - c. Each mark is printed on one line.
 - d. `getTotalMarks()` sums all marks -> `total`.
 - e. Total marks is printed.
 - f. `percentage()` is called:
 - `total` is recalculated via `getTotalMarks()`
 - `percent = (double) total / marks.length`
 - `percent` is printed.
8. Scanner is closed.

3. Algorithm

START

1. Create Scanner object
2. Read name, rollNo from user
3. Create `int` array `marks[3]`; read 3 values into it
4. Create Student object `s(name, rollNo, marks)`
5. CALL `s.displayDetails()`:

```

a. PRINT name.toUpperCase()
b. PRINT RollNo
c. PRINT each value in marks
d. total = CALL getTotalMarks()
   FOR each value in marks: total += value
   RETURN total
e. PRINT total
f. CALL percentage():
   total = CALL getTotalMarks()
   percent = (double) total / marks.length
   PRINT percent
6. Close Scanner
STOP

```

4. UML Class Diagram

```

+-----+
|                               |
|               Student        |
+-----+
| + name : String              |
| # RollNo : int               |
| # marks : int[]              |
+-----+
| + Student(String, int, int[]) |
| ~ getTotalMarks() : int      |
| ~ percentage() : void        |
| ~ displayDetails() : void    |
| + main(String[] args) : void {static}
+-----+

```

Legend: + public, # protected, ~ package-private (default access).

2. StudentPerformaneAnalysia.java — Subclass

Source Code

```

package oops;

import java.util.Scanner;
//import the Student class from the basics package
import basics.Student;

// Student performance analysis class
//Student class inherited by StudentPerformanceAnalysis class using protected
keyword
public class StudentPerformaneAnalysia extends Student {

```

```

// Constructor to initialize student details
public StudentPerformaneAnalysisia(String name, int RollNo, int[] marks) {
    super(name, RollNo, marks);
}

// Method to calculate total marks
int getTotalMarks() {
    int total = 0;
    for (int mark : marks) {
        total += mark;
    }
    return total;
}

// Method to calculate average marks
double getAverageMarks() {
    return (double) getTotalMarks() / marks.length;
}

// Method to find highest marks
int getHighestMarks() {
    int highest = marks[0];
    for (int mark : marks) {
        highest = Math.max(highest, mark);
    }
    return highest;
}

// Method to find lowest marks
int getLowestMarks() {
    int lowest = marks[0];
    for (int mark : marks) {
        lowest = Math.min(lowest, mark);
    }
    return lowest;
}

// Method to calculate percentage and round it to two decimal places
double getPercentage() {
    double percentage = (double) getTotalMarks() / (marks.length * 100) *
100;
    return Math.round(percentage * 100) / 100.0;
}

// Method to assign grade based on percentage
String getGrade() {
    double percentage = getPercentage();
    if (percentage >= 90)
        return "A+";
    else if (percentage >= 80)
        return "A";
    else if (percentage >= 70)
        return "B";
    else if (percentage >= 60)
        return "C";
    else if (percentage >= 50)

```

```

        return "D";
    else
        return "F";
}

// Method to determine pass or fail
boolean isPassed() {
    return getPercentage() >= 50;
}

// Method to display performance remark
String getRemark() {
    switch (getGrade()) {
        case "A+":
            return "Outstanding performance";
        case "A":
            return "Excellent performance";
        case "B":
            return "Very good performance";
        case "C":
            return "Good performance";
        case "D":
            return "Needs improvement";
        default:
            return "Work harder for better results";
    }
}

// Method to display complete student performance report
void displayDetails() {
    String formattedName = name.trim().toUpperCase();
    int total = 0;
    int highest = marks[0];
    int lowest = marks[0];

    // Calculate total, highest and lowest marks in one loop
    for (int mark : marks) {
        total += mark;
        highest = Math.max(highest, mark);
        lowest = Math.min(lowest, mark);
    }

    double average = (double) total / marks.length;
    double percentage = Math.round((double) total / marks.length * 100) /
100.0;

    String grade;
    if (percentage >= 90)
        grade = "A+";
    else if (percentage >= 80)
        grade = "A";
    else if (percentage >= 70)
        grade = "B";
    else if (percentage >= 60)
        grade = "C";
    else if (percentage >= 50)
        grade = "D";
    else

```

```

        grade = "F";

String remark;
switch (grade) {
    case "A+":
        remark = "Outstanding performance";
        break;
    case "A":
        remark = "Excellent performance";
        break;
    case "B":
        remark = "Very good performance";
        break;
    case "C":
        remark = "Good performance";
        break;
    case "D":
        remark = "Needs improvement";
        break;
    default:
        remark = "Work harder for better results";
}

System.out.println("\nStudent Performance Report");
System.out.println("Name: " + formattedName);
System.out.println("Name Length: " + formattedName.length());
System.out.println("Roll Number: " + RollNo);
System.out.print("Marks: ");
for (int mark : marks) {
    System.out.print(mark + " ");
}
System.out.println();
System.out.println("Total Marks: " + total);
System.out.println("Average Marks: " + average);
System.out.println("Highest Marks: " + highest);
System.out.println("Lowest Marks: " + lowest);
System.out.println("Percentage: " + percentage + "%");
System.out.println("Grade: " + grade);
System.out.println("Result: " + (percentage >= 50 ? "Pass" :
"Fail"));
    System.out.println("Remark: " + remark);
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    // Student details
    System.out.print("Enter student name: ");
    String name = sc.nextLine();

    System.out.print("Enter student roll number: ");
    int RollNo = sc.nextInt();

    // Store marks of five subjects in an array
    int[] marks = new int[5];
    System.out.println("Enter marks of five subjects: ");
    for (int i = 0; i < marks.length; i++) {

```

```

        marks[i] = sc.nextInt();
    }

    // Create object and display performance report
    StudentPerformaneAnalysisia st = new StudentPerformaneAnalysisia(name,
RollNo, marks);
    st.displayDetails();

    sc.close();
}
}

```

1. Concept and Exception in Simple Words

Inheritance across packages

`StudentPerformaneAnalysisia` (package `oops`) **extends** `Student` (package `basics`) — this is **inheritance across two different packages**, which only works because:

- `Student` itself is declared `public`.
- Its fields `RollNo` and `marks` are protected — protected members ARE inherited by a subclass even from a different package.
- `name` is `public`, so it's visible everywhere too.
- The constructor calls `super(name, RollNo, marks)` to let the **parent class** initialize the inherited fields.

An important, easy-to-miss detail

`Student`'s methods `getTotalMarks()` and `displayDetails()` have **no access modifier** (package-private / default access). Default-access methods are **only inherited within the same package** — since `StudentPerformaneAnalysisia` lives in a *different* package (`oops`, not `basics`), it **cannot see or override** `Student`'s versions of these methods at all.

So when `StudentPerformaneAnalysisia` declares its own `getTotalMarks()` and `displayDetails()`, these are **brand-new, independent methods** with the same name — **not overrides** of the parent's methods (that's why there's no `@Override` annotation, and correctly so). Both classes simply happen to define a method with a matching name and signature.

Exception in simple words

No exception is thrown or caught in this program.

- `getHighestMarks()` / `getLowestMarks()` / `displayDetails()` all start by reading `marks[0]` — this would throw `ArrayIndexOutOfBoundsException` if `marks` were an **empty array**. The program always creates a 5-element array, so this never happens here.
- `sc.nextInt()` would throw `InputMismatchException` (unchecked) if the user enters non-numeric text for roll number or marks.

- Unlike `Student.java` (which uses `sc.next()` for the name, reading only a single word), this program uses `sc.nextLine()`, allowing the student's **full name with spaces** to be captured correctly.

2. Step-wise Execution

1. Scanner object is created.
2. User enters full name (`sc.nextLine()` – allows spaces).
3. User enters roll number.
4. A 5-element int array 'marks' is created; a loop reads 5 marks.
5. A `StudentPerformaneAnalysia` object 'st' is created.
 - > `super(name, RollNo, marks)` initializes inherited fields `name`, `RollNo`, `marks` via the parent `Student` constructor.
6. `st.displayDetails()` is called (the SUBCLASS's own version):
 - a. `formattedName = name.trim().toUpperCase()`
 - b. `total`, `highest`, `lowest` are computed together in ONE loop over marks (starting `highest = lowest = marks[0]`).
 - c. `average = (double) total / marks.length`
 - d. `percentage = Math.round(total/marks.length*100) / 100.0` (rounded to 2 decimal places)
 - e. grade is decided using if-else on percentage
 - `>=90 -> A+    >=80 -> A    >=70 -> B`
 - `>=60 -> C    >=50 -> D    else -> F`
 - f. remark is decided using a switch on grade
 - g. All values are printed: name, name length, roll number, marks list, total, average, highest, lowest, percentage, grade, Pass/Fail (ternary: `percentage>=50 ? Pass : Fail`), and remark.
7. Scanner is closed.

3. Algorithm

START

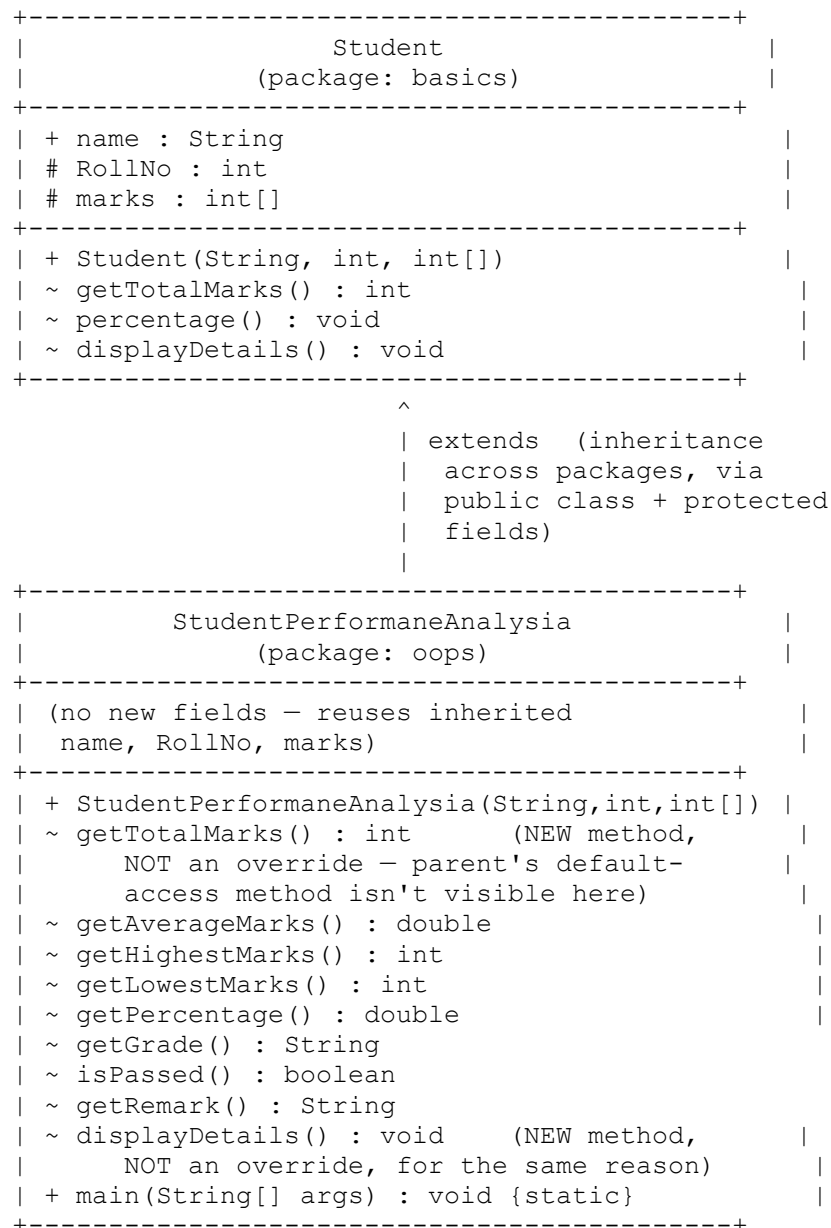
1. Create Scanner object
2. Read name (full line), rollNo from user
3. Create int array `marks[5]`; read 5 values into it
4. Create `StudentPerformaneAnalysia` object `st(name, rollNo, marks)`
 - > parent `Student(name, RollNo, marks)` constructor runs via `super()`
5. CALL `st.displayDetails()`:
 - a. `formattedName = name.trim().toUpperCase()`
 - b. SET `total = 0`, `highest = marks[0]`, `lowest = marks[0]`
 - c. FOR each mark in marks:
 - `total += mark`
 - `highest = MAX(highest, mark)`
 - `lowest = MIN(lowest, mark)`
 - d. `average = total / marks.length` (as double)
 - e. `percentage = ROUND(total/marks.length*100, 2 decimals)`
 - f. IF `percentage >= 90`: grade = "A+"
 - ELSE IF `percentage >= 80`: grade = "A"
 - ELSE IF `percentage >= 70`: grade = "B"
 - ELSE IF `percentage >= 60`: grade = "C"
 - ELSE IF `percentage >= 50`: grade = "D"
 - ELSE: grade = "F"

```

        g. SWITCH on grade -> SET remark string
        h. PRINT name, name length, roll number, marks, total,
           average, highest, lowest, percentage, grade,
           Pass/Fail, remark
    6. Close Scanner
STOP

```

4. UML Class Diagram



Legend: + public, # protected, ~ package-private (default access).

Output:

```
PS C:\Users\Admin\OneDrive\Documents\basicjava> java -cp 'C:\Program Files\Java\jdk-20.0.1\bin\java.exe' --enable-preview --XX:ShowCodeDetails  
InExceptionHandlerMessages' -cp 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'oops.StudentPerformaneAnalysia'  
Enter student name: Susmitha  
Enter student roll number: 208  
Enter marks of five subjects:  
98  
78  
85  
92  
80  
  
Student Performance Report  
Name: SUSMITHA  
Name Length: 8  
Roll Number: 208  
Marks: 98 78 85 92 80  
Total Marks: 433  
Average Marks: 86.6  
Highest Marks: 98  
Lowest Marks: 78  
Percentage: 86.6%  
Grade: A  
Result: Pass  
Remark: Excellent performance  
PS C:\Users\Admin\OneDrive\Documents\basicjava>
```

✦ Result

The Student Performance Analysis System was successfully implemented in Java using OOP concepts, arrays, and built-in packages. The program successfully calculates and displays the student's academic performance, including total marks, average, highest and lowest marks, percentage, grade, result, and performance remark.